Reporte

Equipo 16	
Nombre: Lesly Adamaris Cotlame Nieto Christopher Bethancourt Contreras Francisco Mora Ruiz	**Matrícula**: 2226958 2226948 2226912
Nombre del curso: Componentes de lógica de negocios.	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 4	**Actividad**: Laboratorio guiado. Programa para administrar pedidos de una tienda ficticia TechStore.
Fecha: 28/10/2026	

Bibliografia (Formato APA)

- Beck, K. (2002). Test-driven development: By example. Addison-Wesley Professional.
- Evans, E. (2003). Domain-driven design: Tackling complexity in the heart of software. Addison-Wesley.
- Fowler, M. (2002). Patterns of enterprise application architecture. Addison-Wesley.
- Millett, S., & Tune, N. (2015). Patterns, principles, and practices of domain-driven design. Wrox.
- Vernon, V. (2013). Implementing domain-driven design. Addison-Wesley Professional.

Objetivo:

El estudiante implementará los componentes de lógica de negocio diseñados en U3: entidades DDD, reglas con Specification Pattern, máquinas de estado, Domain Events y APIs REST.

1. Procedimiento y desarrollo

El desarrollo siguió el orden de la práctica guiada: primero el dominio, después las Specifications, la máquina de estados con sus Domain Events y, al final, la API REST. Todas las reglas de negocio están en app/domain.py y app/specifications.py; la API solo las expone por HTTP.

1.1 Entidades y Value Objects

Un Value Object se define por su valor y no cambia después de crearse; una entidad tiene identidad propia y un ciclo de vida. Las clases del dominio son:

Clase	Tipo	Responsabilidad e invariantes que protege
Money	Value Object inmutable	Importe exacto con amount (Decimal) y currency (MXN por omisión). Rechaza float, importes negativos o no finitos (NaN, Infinity) y moneda vacía. add() exige la misma moneda y devuelve un objeto nuevo.
Customer	Clase de datos inmutable	Datos del cliente que el pedido consulta: id, is_vip y active. El identificador no puede estar vacío.
OrderItem	Value Object inmutable	Renglón del pedido: sku, quantity y unit_price. Exige SKU no vacío, cantidad entera mayor que cero y precio de tipo Money. El subtotal se calcula (cantidad × precio), nunca se captura.
OrderStatus	Enumeración	Los seis estados del pedido: DRAFT, CONFIRMED, SHIPPED, DELIVERED, CANCELLED y RETURNED.
DomainEvent	Objeto inmutable	Hecho que ya ocurrió: name, order_id y occurred_at (fecha y hora en UTC).
Order	Entidad y Aggregate Root	Inicia siempre en DRAFT y se identifica por su id. status, items y events son de solo lectura, por lo que el estado solo cambia mediante sus métodos. units suma cantidades (no cuenta renglones) y total suma subtotales sin descuento. add_item() solo funciona en DRAFT, exige precio en MXN y rechaza el artículo si el pedido superaría 20 unidades, sin modificar la colección.

Excepciones de dominio

Excepción	Significado	Ejemplo real	HTTP
DomainException	base de todas las violaciones de reglas de negocio	—	—
InvalidOrderData	datos inválidos o precondición de negocio no cumplida	confirmar O007 vacío; enviar sin guía; Money negativo	422
InvalidOrderTransition	operación incompatible con el estado actual	confirmar O013 por segunda vez; cancelar O014 enviado	409
OrderItemLimitExceeded	se superarían 20 unidades	agregar 2 unidades a O009 (19 + 2 = 21)	422

Los errores técnicos (por ejemplo, una falla de infraestructura) no son excepciones de dominio: no se capturan y la API responde 500, para no hacer pasar un fallo del servidor por un error del usuario.

1.2 Specification Pattern

Las Specifications están en app/specifications.py. Una Specification convierte una condición del negocio en un objeto con el método is_satisfied_by(order), que devuelve True o False y nunca modifica el pedido.

La clase base Specification sobrecarga tres operadores: a & b crea AndSpec (se cumplen ambas partes), a | b crea OrSpec (se cumple al menos una) y ~a crea NotSpec (invierte el resultado). Sobre esa base se implementaron cuatro reglas atómicas:

Specification	Pregunta que responde	Comparación	Ejemplo
MinimumAmountSpec(Decimal("500"))	¿el total supera el mínimo?	order.total.amount > mínimo (estricto)	500.00 → falso; 500.01 → verdadero
VIPCustomerSpec()	¿el cliente es VIP?	order.customer.is_vip	C001 → verdadero; C002 → falso
MinimumUnitsSpec(4)	¿tiene al menos 4 unidades?	order.units >= 4	3 → falso; 4 → verdadero
ActivePromotionSpec()	¿tiene promoción activa?	order.has_promotion	O005 → verdadero

Reglas de descuento del 10 %, 5 % y 0 %

10 %. Cliente VIP, total mayor que 500 MXN, al menos 4 unidades y sin promoción. Combina cuatro condiciones con AND y niega la promoción con NOT:

```
VIP_DISCOUNT_SPEC = (
    MinimumAmountSpec(Decimal("500"))
    & VIPCustomerSpec()
    & MinimumUnitsSpec(4)
    & ~ActivePromotionSpec()
)
```

5 %. Total mayor que 5000 MXN y sin promoción, sin importar si el cliente es VIP:

```
HIGH_AMOUNT_DISCOUNT_SPEC = MinimumAmountSpec(Decimal("5000")) &
~ActivePromotionSpec()
```

0 %. Si el pedido no cumple ninguna de las dos reglas. discount_rate() une ambas reglas con OR y revisa primero la del 10 %, de modo que un pedido que cumple las dos recibe el 10 %:

```
DISCOUNT_ELIGIBLE_SPEC = VIP_DISCOUNT_SPEC | HIGH_AMOUNT_DISCOUNT_SPEC   # OR

def discount_rate(order: Order) -> Decimal:
    if not DISCOUNT_ELIGIBLE_SPEC.is_satisfied_by(order):
        return NO_DISCOUNT_RATE          # Decimal("0.00"): ninguna regla
```

```
    if VIP_DISCOUNT_SPEC.is_satisfied_by(order):
        return VIP_DISCOUNT_RATE          # Decimal("0.10"): tiene prioridad
    return HIGH_AMOUNT_DISCOUNT_RATE      # Decimal("0.05"): solo la del 5 %
```

Revisar primero la regla VIP garantiza que, si un pedido cumple ambas reglas, reciba el 10 %. El importe se calcula aparte: discount_amount() = total × porcentaje, redondeado a centavos con ROUND_HALF_UP; total_with_discount() = total − descuento. order.total nunca cambia.

La siguiente tabla muestra qué ocurre cuando falla una condición:

Regla	Condición	Si no se cumple	Ejemplo real
10 %	cliente VIP	la regla del 10 % falla; puede aplicar el 5 % si el total > 5000	O002 (C002 no VIP) → 0 %
10 %	total > 500 MXN	la regla del 10 % falla; con exactamente 500.00 no hay descuento	VIP, 4 u. y 500.00 (prueba) → 0 %
10 %	al menos 4 unidades	la regla del 10 % falla	O011 (VIP, 3 unidades) → 0 %
10 % y 5 %	sin promoción activa	fallan ambas reglas	O005 (promoción) → 0 %
5 %	total > 5000 MXN	la regla del 5 % falla	O002 (1050.00) → 0 %
ambas	se cumplen las dos	se aplica el 10 % por prioridad	VIP, 4 u. y 8000.00 (prueba) → 10 %

1.3 Máquina de estados

Todo pedido nace en DRAFT. Las únicas transiciones permitidas son:

Transición	Método	Precondiciones (en orden)	Error si falla	Evento
DRAFT → CONFIRMED	confirm()	estado DRAFT; al menos un artículo; cliente activo	409 si no está en DRAFT; 422 si está vacío o el cliente está inactivo	OrderConfirmed
CONFIRMED → SHIPPED	ship(tracking_number)	estado CONFIRMED; guía no vacía	409 / 422	OrderShipped
CONFIRMED → CANCELLED	cancel(reason)	estado CONFIRMED; motivo no vacío	409 / 422	OrderCancelled
SHIPPED → DELIVERED	deliver()	estado SHIPPED	409	OrderDelivered
SHIPPED → RETURNED	return_order()	estado SHIPPED	409	OrderReturned

La Figura 1 muestra el ciclo de vida del pedido con la notación operación() [condición] / evento registrado. Las notas amarillas resumen las precondiciones, los rechazos y las reglas comunes.

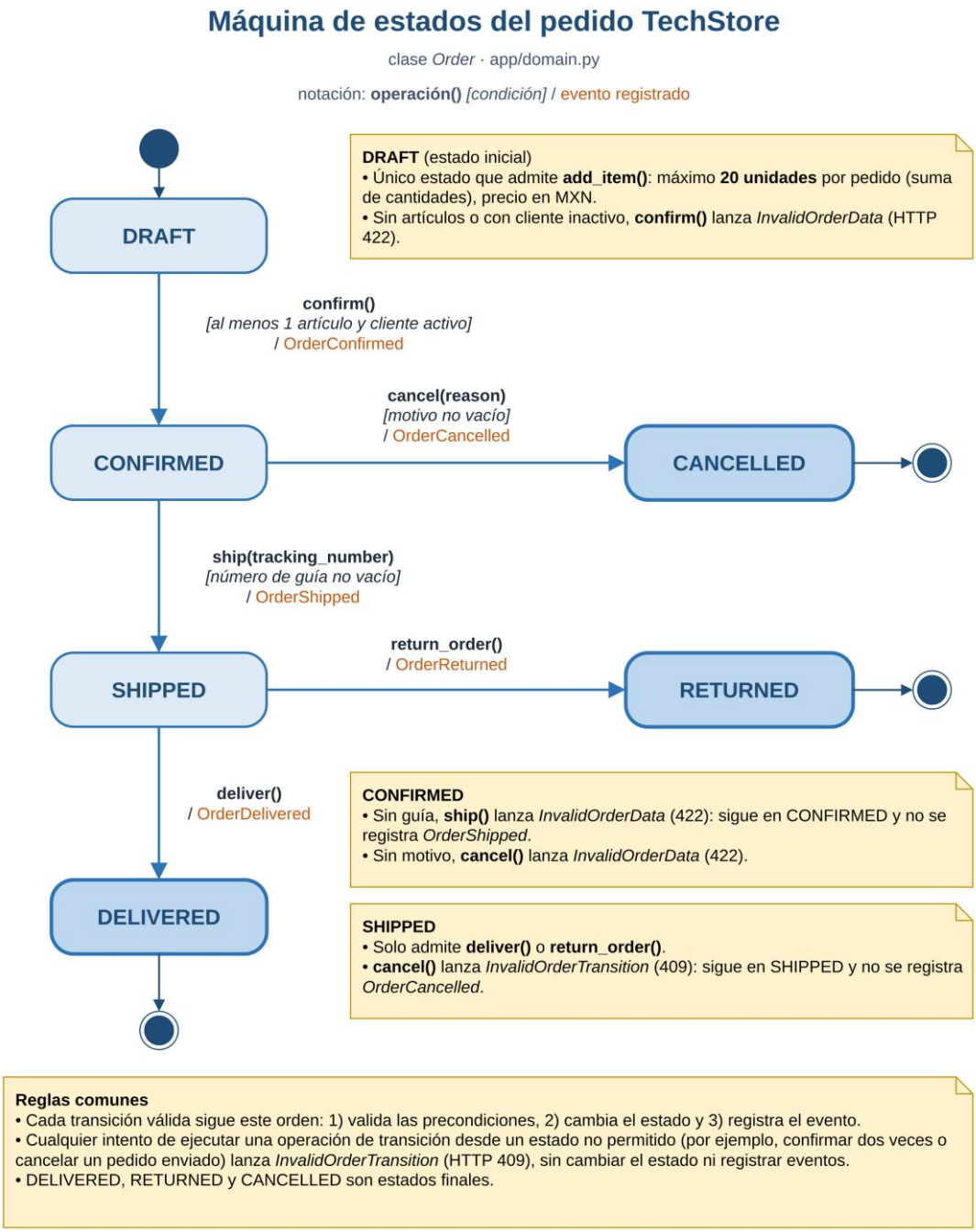


Figura 1. *Máquina de estados del pedido TechStore. Elaborada en Draw.io (docs/diagramas/Diagrama_Estados_TechStore.drawio).*

DELIVERED, RETURNED y CANCELLED son estados finales. Como indica la regla común del diagrama, cada transición válida sigue este orden: 1) valida las precondiciones, empezando por el estado con _require_status(...); 2) cambia el estado, y 3) registra el evento.

Cualquier intento de ejecutar una operación de transición desde un estado no permitido (por ejemplo, confirmar dos veces, escenario E12, o cancelar un pedido enviado, escenario E13) lanza InvalidOrderTransition (HTTP 409), sin cambiar el estado ni registrar eventos. Lo mismo ocurre al enviar un pedido en DRAFT, al entregar uno que no se ha enviado o al intentar cualquier operación de transición desde DELIVERED, RETURNED o CANCELLED. La prueba test_matriz_de_transiciones recorre las 30 combinaciones posibles (6 estados × 5 operaciones): las 5 válidas cambian el estado y registran su evento, y las otras 25 se rechazan sin alterar el pedido.

1.4 Domain Events

Un Domain Event comunica que un hecho del negocio ya ocurrió; por eso su nombre está en pasado. Se registra en Order.events al final de cada transición válida (validar → cambiar estado → registrar). La práctica pide registrar los eventos, no publicarlos: no se envían correos ni se actualiza inventario.

OrderConfirmed

Representa que el pedido quedó confirmado. Lo genera confirm() cuando el pedido estaba en DRAFT, tenía artículos y el cliente estaba activo. No se registra si el pedido está vacío (O007), si el cliente está inactivo (O003) o si ya estaba confirmado (segunda confirmación de O013).

OrderShipped

Representa que el pedido salió con un número de guía. Lo genera ship(tracking_number) cuando el pedido estaba en CONFIRMED y la guía no está vacía; la guía se guarda en tracking_number. No se registra si falta la guía (E11, O012) ni si el pedido no está confirmado.

OrderCancelled

Representa que un pedido confirmado se canceló por un motivo. Lo genera cancel(reason) cuando el pedido estaba en CONFIRMED y el motivo no está vacío; el motivo se guarda en cancellation_reason. No se registra si el pedido ya fue enviado (E13, O014) ni si falta el motivo.

OrderDelivered

Representa que el cliente recibió el pedido. Lo genera deliver() cuando el pedido estaba en SHIPPED (E15, O016). Desde cualquier otro estado se lanza InvalidOrderTransition y no se registra.

OrderReturned

Representa que el pedido enviado fue devuelto. Lo genera return_order() cuando el pedido estaba en SHIPPED (E14, O015). Desde cualquier otro estado se lanza InvalidOrderTransition y no se registra.

1.5 API REST y FastAPI

app/api.py expone el dominio con FastAPI. Los endpoints son delgados: llaman a OrderService (app/service.py), que busca el pedido en memoria y ejecuta el método de negocio, y después convierten el pedido en JSON. Para mostrar el descuento, la API consulta las mismas Specifications del dominio; no repite las reglas ni cambia el estado directamente. OrderNotFound no hereda de DomainException, porque un pedido inexistente no es una regla de negocio violada.

Endpoints implementados

Método	Endpoint	Finalidad	Parámetros	Respuestas
GET	/orders	listar los pedidos en memoria (consulta adicional)	—	200
GET	/orders/{order_id}	consultar un pedido con su descuento	order_id (ruta)	200 · 404
POST	/orders/{order_id}/confirm	confirmar (DRAFT → CONFIRMED)	order_id	200 · 404 · 409 · 422
POST	/orders/{order_id}/ship	enviar (CONFIRMED → SHIPPED)	order_id; cuerpo {"tracking_number": "MX-123"}	200 · 404 · 409 · 422
POST	/orders/{order_id}/cancel	cancelar (CONFIRMED → CANCELLED)	order_id; cuerpo {"reason": "..."}	200 · 404 · 409 · 422
POST	/orders/{order_id}/deliver	entregar (SHIPPED → DELIVERED), adicional	order_id	200 · 404 · 409
POST	/orders/{order_id}/return	devolver (SHIPPED → RETURNED), adicional	order_id	200 · 404 · 409

Códigos HTTP

Código	Significado	Excepción	Ejemplo del proyecto
200 OK	la consulta o la operación se realizó	—	POST /orders/O001/confirm → "status": "CONFIRMED"
404 Not Found	el pedido no existe	OrderNotFound	POST /orders/O999/confirm → «No existe el pedido O999.»
409 Conflict	la operación choca con el estado actual	InvalidOrderTransition	segunda confirmación de O013; cancelar O014 enviado
422 Unprocessable Entity	datos o precondiciones de negocio inválidos, o JSON con formato incorrecto	InvalidOrderData, OrderItemLimitExceeded, RequestValidationError	confirmar O007 vacío; enviar O012 sin guía; confirmar O003 con cliente inactivo

Todos los errores usan el mismo formato JSON, por ejemplo:

```
{"error": "InvalidOrderData",
 "message": "No se puede confirmar el pedido O007: no tiene artículos.",
 "order_id": "O007"}
```

Swagger y OpenAPI

FastAPI genera el contrato OpenAPI a partir de los endpoints y de los modelos Pydantic. Se publica en /openapi.json y Swagger UI lo presenta en /docs, donde cada endpoint se prueba con Try it out y Execute. La Figura 2 muestra los siete endpoints; las operaciones ejecutadas en Swagger están en la sección 5.

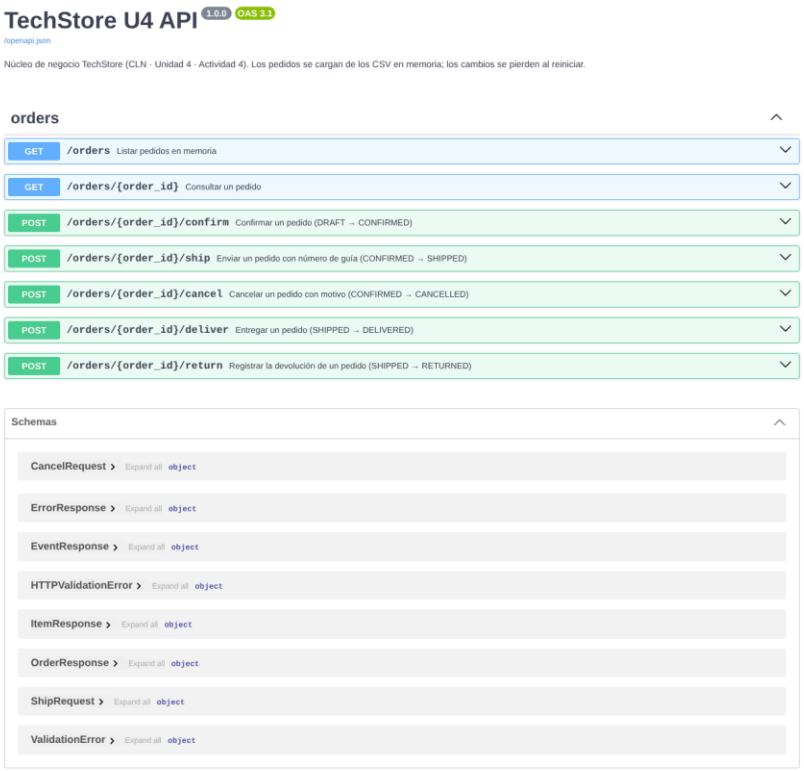


Figura 2. *Swagger UI de TechStore U4 con los siete endpoints y los modelos de petición y respuesta. Captura del entorno de desarrollo.*

Organización de responsabilidades: El dominio no depende de FastAPI ni de archivos; cargar_csv.py convierte los CSV en objetos del dominio; el servicio coordina los casos de uso, y la API traduce las peticiones y las excepciones. Por eso la API, ejecutar_escenarios.py y las pruebas obtienen exactamente el mismo comportamiento.

2. Ejemplos de funcionamiento

2.1 Caso O001: carga, total y descuento del 10 %

O001 pertenece a C001 (Ana Torres, VIP y activa), no tiene promoción y tiene dos renglones: SKU001 × 2 a 600.00 MXN y SKU002 × 2 a 450.00 MXN. El cargador crea el pedido en DRAFT y agrega los dos artículos con add_item().

Importes: Subtotales: 2 × 600.00 = 1,200.00 y 2 × 450.00 = 900.00. Unidades: 2 + 2 = 4 (2 renglones, pero 4 unidades). Total original: 2,100.00 MXN.

Descuento. discount_rate(orders["O001"]), donde orders es el diccionario que devuelve load_fixture(), evalúa las condiciones:

Condición	Valor de O001	¿Se cumple?
MinimumAmountSpec(500)	2100.00 > 500	sí
VIPCustomerSpec()	C001 es VIP	sí
MinimumUnitsSpec(4)	4 ≥ 4	sí
~ActivePromotionSpec()	sin promoción	sí
HIGH_AMOUNT_DISCOUNT_SPEC	2100.00 > 5000	no

Se cumple la regla del 10 %: el descuento es 2,100.00 × 0.10 = 210.00 MXN y el total con descuento, 1,890.00 MXN. orders["O001"].total sigue valiendo 2,100.00, porque evaluar una Specification no modifica el pedido.

Confirmación: POST /orders/O001/confirm cambia el pedido a CONFIRMED y registra OrderConfirmed; la API responde 200 con esos importes (Figura 6). Una segunda confirmación responde 409 (Figura 8).

2.2 Caso O014: por qué no se cancela después del envío

O014 pertenece a C002 (activo) y tiene 1 unidad de SKU001 (600.00 MXN). El escenario E13 ejecuta:

Paso	Operación	Validación	Resultado	Eventos acumulados
1	confirm()	DRAFT, con artículos, cliente activo	CONFIRMED	OrderConfirmed
2	ship("MX-GUIA-0001")	CONFIRMED y guía no vacía	SHIPPED	OrderConfirmed, OrderShipped
3	cancel("Solicitud del cliente")	_require_status(CONFIRMED) falla: está en SHIPPED	rechazo InvalidOrderTransition	sin cambios: OrderConfirmed, OrderShipped

cancel() empieza verificando el estado con _require_status(...). Como O014 está en SHIPPED, la excepción se lanza en esa primera validación: el pedido conserva el estado SHIPPED y no se registra OrderCancelled. La razón es de negocio: el paquete ya salió del almacén, y desde SHIPPED solo tiene sentido entregarlo (deliver()) o registrar su devolución (return_order()). En la API, el intento responde 409 Conflict.

3. Pruebas y validaciones

3.1 Resultados de la ejecución

La suite completa se ejecutó con python -m pytest -q en dos entornos, ambos con Python 3.13.16:

- Computadora del equipo (Windows, PowerShell en VS Code, entorno `.venv`): 232 passed, 1 warning in 2.50s (Figura 4). La versión de Python se muestra en la Figura 3.
- Entorno de desarrollo (Linux): 232 passed, con FastAPI 0.116.1, Starlette 0.47.3, Pydantic 2.13.5, pytest 9.1.1 y httpx 0.28.1.

Archivo de pruebas	Objetivo	Pruebas	Resultado registrado
tests/test_domain.py	Money, Customer, OrderItem, Order, límite de 20, las 30 combinaciones de estado × operación y los eventos	133	aprobadas
tests/test_specifications.py	cada Specification, tablas de verdad AND/OR/NOT, fronteras, prioridad, promoción y redondeo	40	aprobadas
tests/test_api.py	200, 404, 409, 422, envío sin guía, cancelación desde SHIPPED, aislamiento, error 500 y OpenAPI	23	aprobadas
tests/test_escenarios.py	E01–E15 con los CSV originales	17	aprobadas
tests/test_datos.py	CSV sin modificar (SHA-256 de los bytes almacenados), número de registros y cargador	19	aprobadas
Total		232	232 passed (Windows y Linux)

El número de pruebas por archivo se obtuvo en el entorno de desarrollo; la captura de Windows muestra el total, que coincide con la suma (232). Además, con Python 3.11.17 (versión mínima de la práctica) se ejecutaron en Linux las pruebas que no requieren FastAPI: 209 passed.

La advertencia de ambas ejecuciones es un DeprecationWarning de starlette/testclient.py (un alias obsoleto de anyio). Proviene de una dependencia, no del proyecto, y no afecta a ninguna prueba.

3.2 Qué verifican las pruebas

Dinero e invariantes. Money rechaza float, importes negativos, NaN, Infinity y monedas distintas; OrderItem rechaza SKU vacío y cantidades no positivas; Order acepta 20 unidades y rechaza 21 sin modificar sus artículos, solo admite artículos en DRAFT y no permite cambiar status, items ni events desde fuera.

Specifications y descuentos. Cada Specification por separado, las tablas de verdad de AND, OR y NOT, las fronteras (500.00 y 500.01; 5000.00 y 5000.01; 3 y 4 unidades), la prioridad del 10 %, la promoción que bloquea ambos descuentos y el redondeo a centavos.

Transiciones y Domain Events. Las 30 combinaciones de estado × operación, los cinco eventos, su fecha en UTC y su orden. En cada rechazo se compara una «fotografía» del pedido antes y después, para demostrar que no cambió el estado ni se agregaron eventos.

Respuestas HTTP de la API. 200 en consultas y operaciones válidas, 404 para un pedido inexistente, 409 para transiciones no permitidas y 422 para datos inválidos (pedido vacío, cliente inactivo, envío sin guía), siempre con el mismo formato JSON de error.

Integridad de los CSV y escenarios. La huella SHA-256 de cada CSV se compara con la de los archivos originales, byte a byte; se verifica el número de registros, y los escenarios E01–E15 se ejecutan con los datos originales.

3.3 Verificaciones adicionales realizadas

Para comprobar la importancia del orden de las operaciones, se analiza qué ocurriría si un Domain Event se registrara antes de completar las validaciones. Esto podría registrar un evento de una operación que finalmente fuera rechazada. Por eso, el código primero valida las precondiciones, después cambia el estado del pedido y finalmente registra el evento. Si la operación falla, el estado no cambia y no se registra el evento correspondiente.

Verificación	Resultado observado
> cambiado a >= en MinimumAmountSpec	3 pruebas fallan. E09 sigue dando 0 %, porque O010 tampoco cumple las demás condiciones de las reglas.
units cuenta renglones (len(self._items)) en lugar de sumar cantidades	22 pruebas fallan. E01 da 0 % y E08 acepta el artículo: 13/15 escenarios.
Copia limpia del proyecto, sin cambios	16 pedidos cargados, 232 pruebas aprobadas y 15/15 escenarios.

4. Bitácora completa de los escenarios E01–E15

La bitácora se obtuvo con python ejecutar_escenarios.py --bitacora, que lee datos/escenarios.csv y vuelve a cargar los CSV para cada caso, de modo que un escenario no afecta al siguiente. Los 15 escenarios cumplen su objetivo (15/15) en el entorno de desarrollo y en la computadora del equipo (Figura 5). La operación y el objetivo de cada caso se copian tal como aparecen en el CSV.

Caso	Estado inicial	Operación	Resultado real	Explicación
E01	O001 DRAFT, 4 u., 2100.00 MXN	evaluar_descuento	10 % (objetivo 10%). Descuento 210.00 MXN; total con descuento 1890.00 MXN. Estado final: DRAFT. Sin eventos.	Regla del 10 %: VIP sí, 2100.00 > 500 sí, 4 ≥ 4 unidades sí, sin promoción sí → se cumple. Evaluar una Specification no cambia el pedido ni registra eventos.
E02	O002 DRAFT, 2 u., 1050.00 MXN	evaluar_descuento	0 % (objetivo 0%). Total 1050.00 MXN sin descuento. Estado final: DRAFT. Sin eventos.	C002 no es VIP y tiene 2 unidades: falla la regla del 10 %. 1050.00 no supera 5000: falla la del 5 %.
E03	O003 DRAFT, 4 u., 2100.00 MXN	confirmar	Rechazo (objetivo rechazo). InvalidOrderData (422). Estado final: DRAFT. No se registró OrderConfirmed.	El cliente C003 está inactivo. confirm() lo valida antes de cambiar el estado, así que el pedido queda intacto.
E04	O005 DRAFT, 4 u., 2100.00 MXN	evaluar_descuento	0 % (objetivo 0%). Estado final: DRAFT. Sin eventos.	O005 tiene promoción activa: ~ActivePromotionSpec() falla en ambas reglas, aunque el cliente es VIP, con 4 unidades y 2100.00 MXN.

Caso	Estado inicial	Operación	Resultado real	Explicación
E05	O006 DRAFT, 1 u., 18000.00 MXN	evaluar_descuento	5 % (objetivo 5%). Descuento 900.00 MXN; total con descuento 17100.00 MXN. Estado final: DRAFT. Sin eventos.	No es VIP y tiene 1 unidad (falla el 10 %), pero 18000.00 > 5000 y no tiene promoción: se cumple la regla alternativa del 5 % (OR).
E06	O007 DRAFT, 0 u., 0.00 MXN	confirmar	Rechazo (objetivo rechazo). InvalidOrderData (422). Estado final: DRAFT. No se registró OrderConfirmed.	Un pedido sin artículos no puede confirmarse.
E07	O008 DRAFT, 20 u., 2400.00 MXN	confirmar	Éxito (objetivo exito). Estado final: CONFIRMED. Evento OrderConfirmed.	20 unidades están dentro del límite (≤ 20); el pedido tiene artículos y el cliente está activo.
E08	O009 DRAFT, 19 u., 2280.00 MXN	agregar SKU003 x2	Rechazo (objetivo rechazo). OrderItemLimitExceeded (422). Estado final: DRAFT, con 19 unidades. Sin eventos.	19 + 2 = 21 > 20. El límite suma cantidades, no renglones; el artículo no se agrega. add_item() no registra eventos.
E09	O010 DRAFT, 2 u., 500.00 MXN	evaluar_descuento	0 % (objetivo 0%). Estado final: DRAFT. Sin eventos.	El umbral es estricto: 500.00 no es mayor que 500. Además, el cliente no es VIP ni el pedido tiene 4 unidades.
E10	O011 DRAFT, 3 u., 1500.00 MXN	evaluar_descuento	0 % (objetivo 0%). Estado final: DRAFT. Sin eventos.	C011 es VIP y 1500.00 > 500, pero solo hay 3 unidades y la regla exige 4. 1500.00 tampoco supera 5000.

Caso	Estado inicial	Operación	Resultado real	Explicación
E11	O012 DRAFT, 1 u., 850.00 MXN	confirmar y enviar sin guia	Rechazo del envío (objetivo rechazo envio). Confirmación válida con OrderConfirmed; ship() lanza InvalidOrderData (422). Estado final: CONFIRMED. No se registró OrderShipped.	La guía es obligatoria para enviar. El pedido conserva CONFIRMED y un solo evento.
E12	O013 DRAFT, 1 u., 600.00 MXN	confirmar dos veces	Rechazo de la segunda confirmación (objetivo rechazo segunda). La primera registra OrderConfirmed; la segunda lanza InvalidOrderTransition (409). Estado final: CONFIRMED. No se registró un segundo OrderConfirmed.	confirm() solo se permite desde DRAFT.
E13	O014 DRAFT, 1 u., 600.00 MXN	confirmar, enviar y cancelar	Rechazo de la cancelación (objetivo rechazo cancelacion). Eventos OrderConfirmed y OrderShipped; cancel() lanza InvalidOrderTransition (409). Estado final: SHIPPED. No se registró OrderCancelled.	Solo se cancela desde CONFIRMED; un pedido enviado solo puede entregarse o devolverse (sección 2.2).
E14	O015 DRAFT, 1 u., 450.00 MXN	confirmar, enviar y devolver	Éxito (objetivo exito). Estado final: RETURNED. Eventos OrderConfirmed, OrderShipped y OrderReturned.	Tres transiciones válidas; cada una registra su evento después de cambiar el estado.
E15	O016 DRAFT, 1 u., 2100.00 MXN	confirmar, enviar y entregar	Éxito (objetivo exito). Estado final: DELIVERED. Eventos OrderConfirmed, OrderShipped y OrderDelivered.	Recorre el camino normal del pedido hasta el estado final DELIVERED.

5. Evidencias

Las figuras de esta sección son capturas reales tomadas por el equipo y no se editaron. Las de la terminal muestran PowerShell en VS Code sobre Windows, con el entorno .venv del proyecto activado; las de Swagger UI muestran el servidor local http://127.0.0.1:8000 y, en el encabezado date, la hora de cada respuesta. No se incluye la captura opcional de GitHub porque no se cuenta con una captura del repositorio publicado.

5.1 Evidencia 1 · Carga de los 16 pedidos originales

python --version muestra Python 3.13.16 y python cargar_csv.py imprime los 16 pedidos en DRAFT con sus unidades y su total: O001 con 4 unidades y 2100.00 MXN, y O007 con 0 unidades. Esto demuestra que el cargador original funciona con las clases implementadas sin modificarlo. Los identificadores empiezan con la letra O (O001 a O016), igual que en datos/pedidos.csv; en la fuente de la terminal la O es redonda y sin marca interior, mientras que el cero lleva una barra, por lo que a primera vista O001 puede parecer 0001.

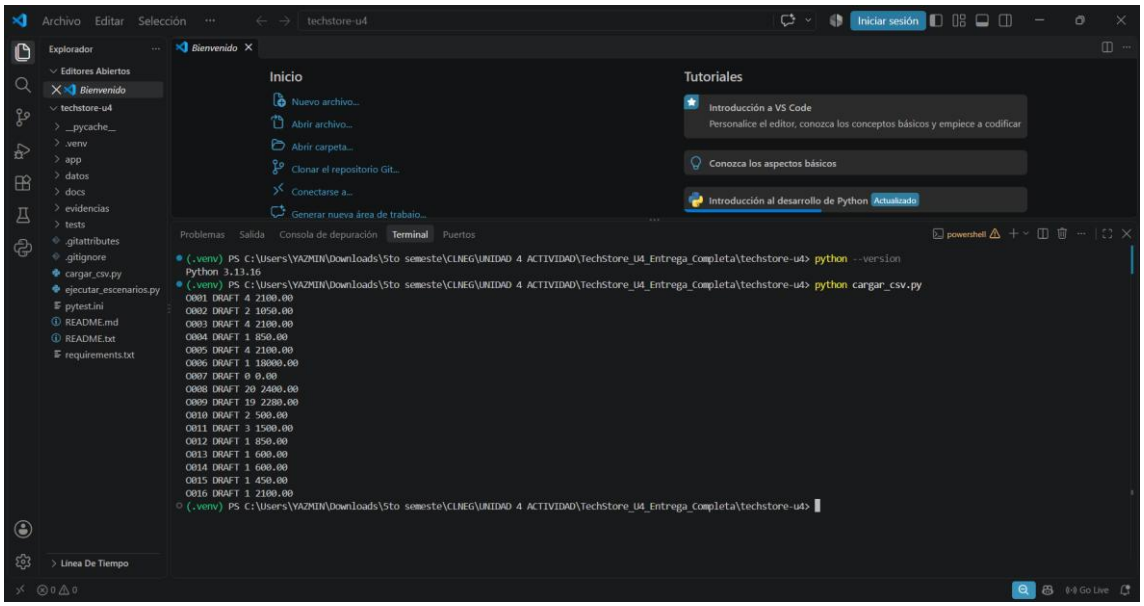


Figura 3. *Ejecución de python cargar_csv.py en Windows: 16 pedidos cargados en estado DRAFT.*

5.2 Evidencia 2 · Ejecución de las pruebas automatizadas

python -m pytest -q termina con 232 passed, 1 warning in 2.50s. La advertencia es el DeprecationWarning de Starlette explicado en la sección 3.1.

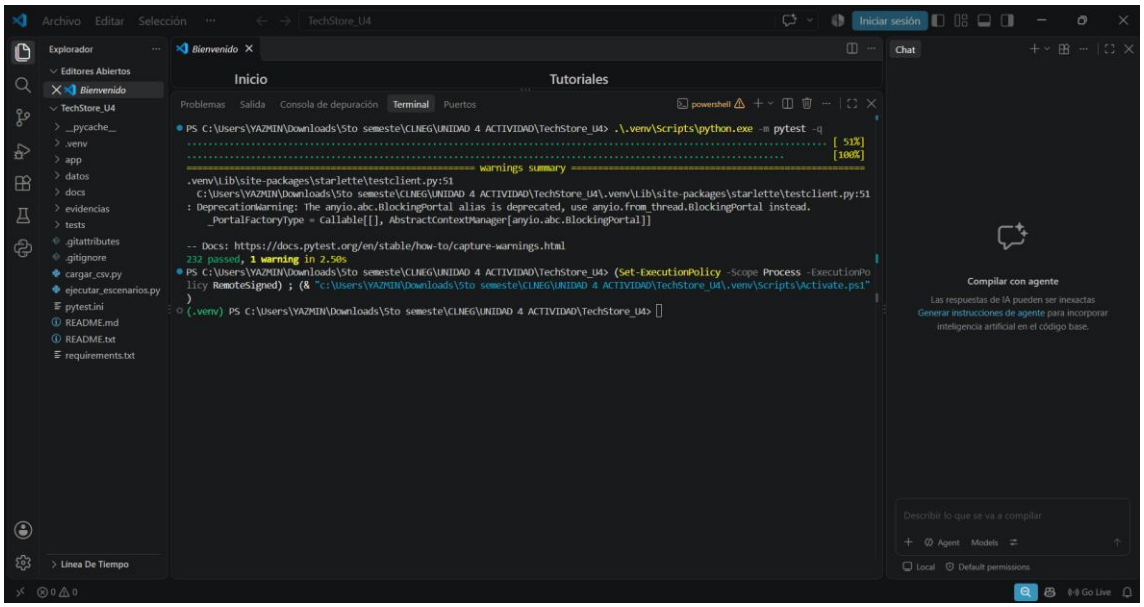


Figura 4. *Ejecución de python -m pytest -q en Windows: 232 pruebas aprobadas.*

5.3 Evidencia 3 · Ejecución de los escenarios E01–E15

python ejecutar_escenarios.py --bitacora muestra 15 líneas [OK] y el mensaje «15/15 escenarios cumplen su objetivo.» En la parte superior se ve el final de una ejecución anterior de pytest (215 passed), previa a las pruebas de Customer. La explicación de cada caso está en la bitácora de la sección 4.

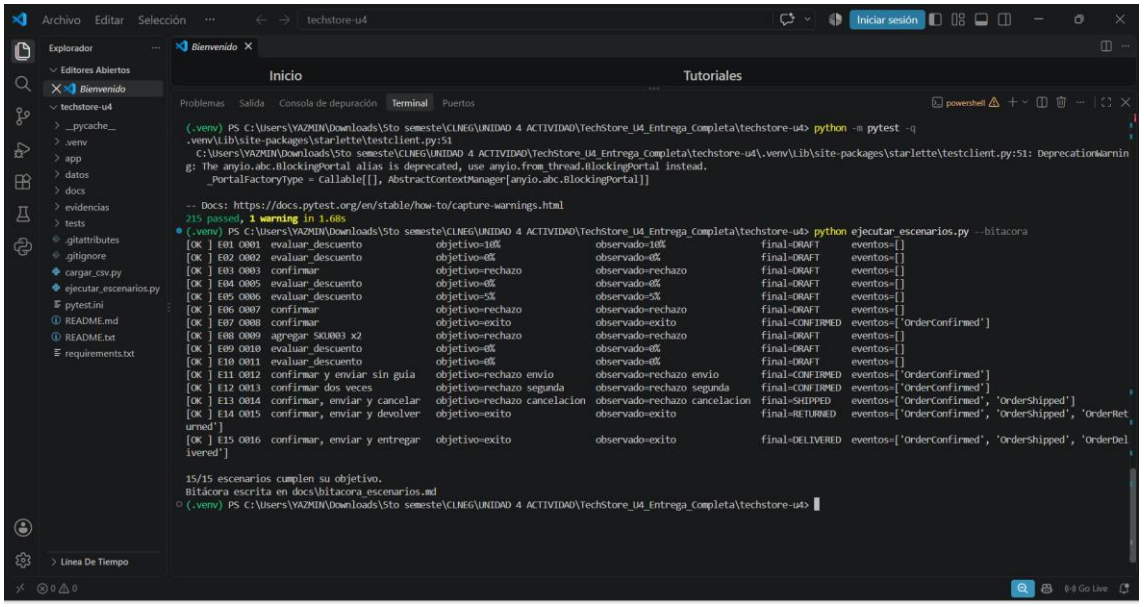


Figura 5. *Ejecución de python ejecutar_escenarios.py --bitacora en Windows: 15/15 escenarios cumplen su objetivo.*

5.4 Evidencia 4 · Operación exitosa en Swagger

POST /orders/O001/confirm respondió 200 con "status": "CONFIRMED", 4 unidades, total de 2100.00 MXN, discount_rate de 0.10, descuento de 210.00 MXN y total con descuento de 1890.00 MXN, que coinciden con el cálculo de la sección 2.1.

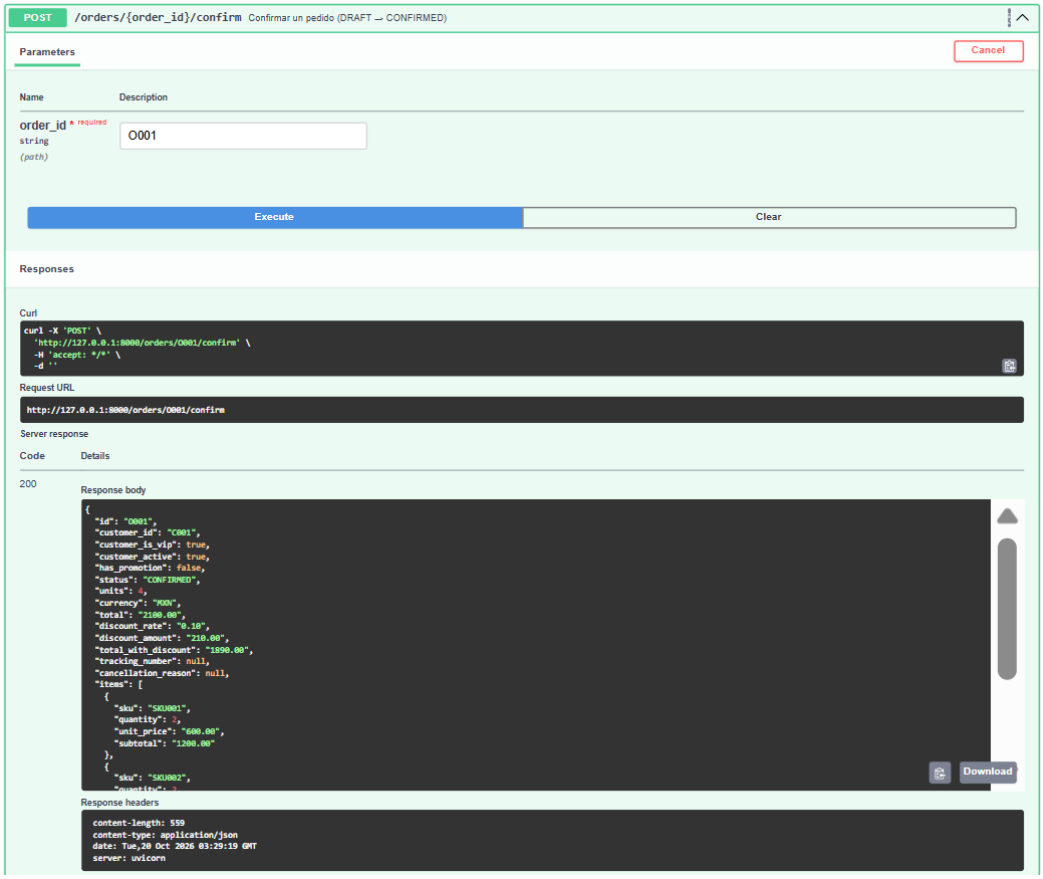


Figura 6. *Swagger UI: confirmación exitosa de O001 (HTTP 200, estado CONFIRMED).*

5.5 Evidencia 5 · Operaciones rechazadas en Swagger

Las Figuras 7 a 9 muestran los tres tipos de rechazo. En todos, la API responde con el mismo formato JSON (error, message y order_id):

- **404:** POST /orders/O999/confirm → OrderNotFound, «No existe el pedido O999.» (Figura 7).
- **409:** segunda confirmación de O001 → InvalidOrderTransition, «No se puede confirmar el pedido O001: está en CONFIRMED y esta operación solo se permite desde DRAFT.» (Figura 8).
- **422:** confirmación de O007 → InvalidOrderData, «No se puede confirmar el pedido O007: no tiene artículos.» (Figura 9).

Los pedidos se guardan en memoria y se vuelven a cargar cada vez que arranca el servidor. Por eso la confirmación exitosa de O001 (Figura 6) tiene una hora posterior a la del rechazo 409 (Figura 8): se tomó en una ejecución posterior del servidor, cuando O001 había vuelto a DRAFT.

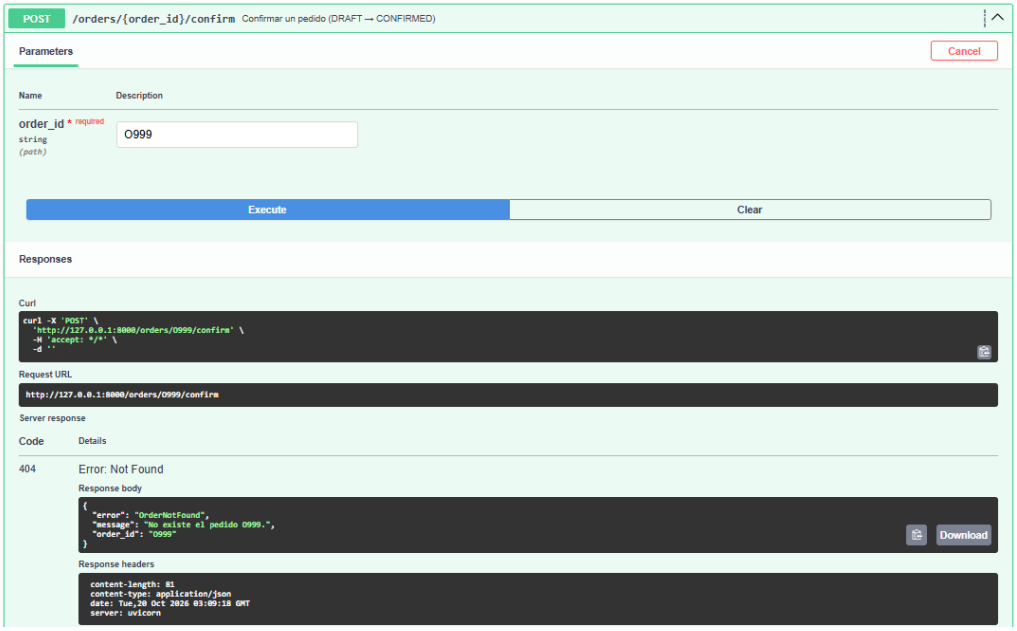


Figura 7. *Swagger UI: confirmación de O999, pedido inexistente (HTTP 404, OrderNotFound).*

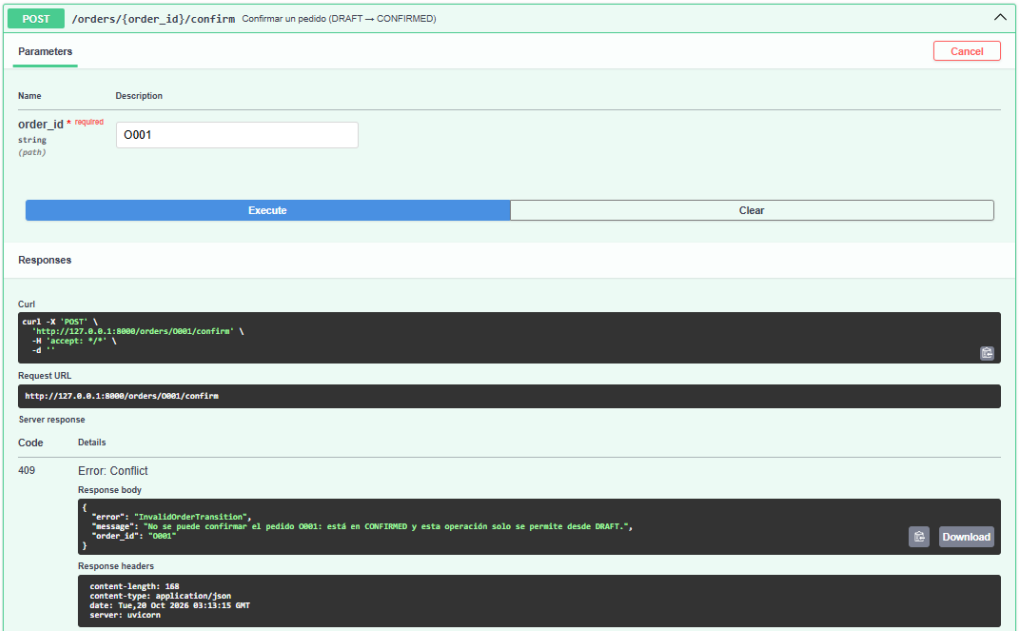


Figura 8. *Swagger UI: segunda confirmación de O001 (HTTP 409, InvalidOrderTransition).*

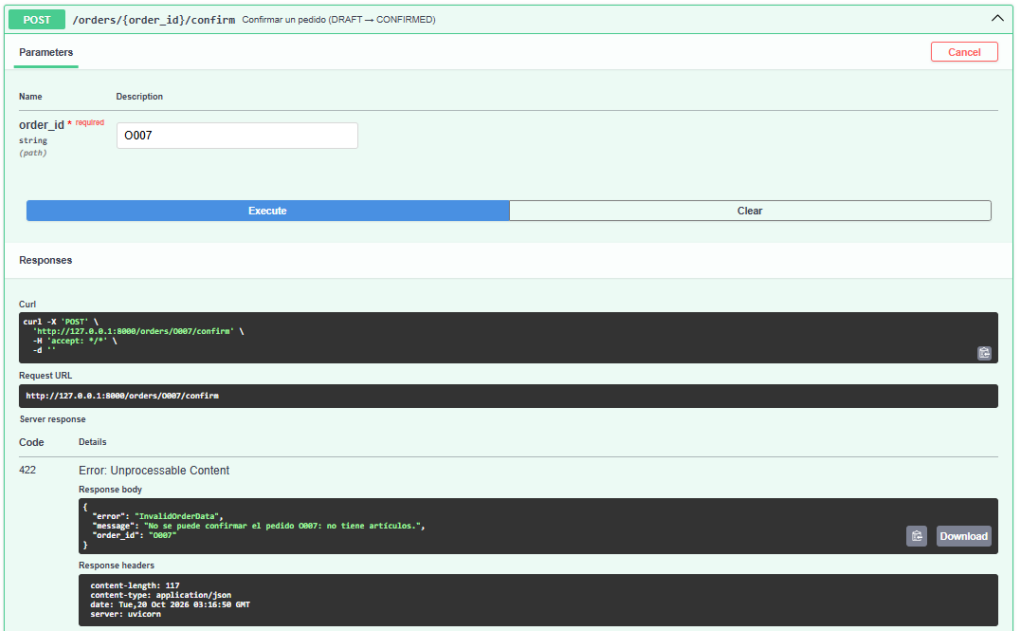


Figura 9. *Swagger UI: confirmación de O007, pedido sin artículos (HTTP 422, InvalidOrderData).*

6. Reflexión sobre los errores encontrados

Se documentan tres errores que ocurrieron realmente durante la preparación del proyecto. El primero estaba en una prueba del proyecto. Los otros dos ocurrieron en el entorno temporal en la nube que se usó para desarrollar y probar el proyecto, no en la computadora del equipo, y sus soluciones no forman parte del código entregado.

6.1 Error 1 · Las huellas de los CSV no coincidían (error en una prueba del proyecto)

Aspecto	Descripción
Qué sucedió	La primera ejecución de python -m pytest -q dio 5 failed, 210 passed: fallaron las cinco pruebas test_csv_originales_sin_modificar.
Cómo se detectó y causa	pytest mostró dos huellas SHA-256 distintas para cada CSV. Los CSV del profesor usan saltos de línea de Windows (CRLF); las huellas esperadas se calcularon sobre los archivos tal cual, pero la prueba convertía CRLF en LF antes de calcular la huella. La lógica de negocio era correcta.
Solución	La prueba compara los bytes tal como están almacenados, sin normalizar, contra las huellas del paquete original, y .gitattributes (*.csv -text) impide que Git modifique los CSV. Resultado en ese momento: 215 passed (232 en la validación final, tras agregar las pruebas de Customer). Como contraprueba, convertir clientes.csv a LF en una copia hace fallar la prueba (1 failed, 18 passed).
Aprendizaje	Dos archivos que se ven iguales en el editor pueden ser distintos byte por byte. Una prueba debe comparar exactamente lo mismo que se calculó, y su resultado esperado no se cambia sin entender primero la causa.

6.2 Error 2 · Swagger UI aparecía en blanco (problema del entorno)

Aspecto	Descripción
Qué sucedió	En el entorno temporal de desarrollo, con el servidor funcionando (/docs respondía 200), la página de Swagger UI aparecía en blanco al abrirla con un navegador automatizado.
Cómo se detectó y causa	La página /docs de FastAPI descarga el JavaScript y el CSS de Swagger UI desde cdn.jsdelivr.net, y ese entorno bloqueaba el acceso a ese sitio.
Solución	Solo para tomar la captura de la Figura 2 en ese entorno, el navegador recibió una copia local de los archivos de Swagger UI. Esa copia y esa configuración no están en el proyecto entregado: app/api.py usa la página /docs estándar de FastAPI. En la computadora del equipo, Swagger UI funcionó sin ningún ajuste (Figuras 6 a 9).
Aprendizaje	/docs necesita Internet aunque la API sea local; /openapi.json no. Si /docs aparece en blanco, la API no está fallando: falta la conexión al CDN.

6.3 Error 3 · No se podía instalar FastAPI desde PyPI (problema del entorno)

Aspecto	Descripción
Qué sucedió	En el entorno temporal de desarrollo, pip install fastapi pytest httpx terminó con «No matching distribution found for fastapi».
Cómo se detectó y causa	pip mostró errores de conexión y después 403 Forbidden desde el proxy de ese entorno, que bloqueaba el acceso a pypi.org. No era un problema del proyecto, y no hay evidencia de que ocurriera en la computadora del equipo.
Solución	En ese entorno se usó el código fuente oficial de FastAPI 0.116.1 y Starlette 0.47.3 publicado en GitHub, y pytest y httpx de una caché local. El proyecto no incluye esas copias: requirements.txt solo fija las versiones con las que se aprobaron las 232 pruebas.
Aprendizaje	Las versiones de las dependencias forman parte del proyecto: fijarlas permite reproducir el mismo entorno en otra computadora.

7. Preguntas del reporte y respuestas

Pregunta 1. ¿Por qué se utiliza Decimal en lugar de float para precios?

float guarda los números en binario y muchos decimales no tienen una representación exacta: en Python, 0.1 + 0.2 da 0.30000000000000004. En dinero, esos errores se acumulan, producen centavos de más o de menos y pueden alterar las comparaciones con los umbrales (un total que debería ser exactamente 500 podría superar > 500). Decimal("600.00") representa exactamente el valor escrito. Por eso Money rechaza float y los precios se construyen desde texto: Decimal("600.00"), no Decimal(600.00).

Pregunta 2. ¿Cuál es la diferencia entre contar renglones y sumar unidades?

Un renglón es una línea del pedido (un SKU con su cantidad); las unidades son la suma de las cantidades. O001 tiene 2 renglones (SKU001 ×2 y SKU002 ×2) pero 4 unidades, y O008 tiene 1 renglón con 20 unidades. Las reglas del laboratorio («máximo 20 unidades» y «al menos 4 unidades») hablan de unidades. Contar renglones permitiría un solo renglón con 500 piezas y le negaría el descuento a O001. Por eso Order.units usa sum(item.quantity ...) y no len(items).

Pregunta 3. ¿Qué ventaja tiene encapsular una regla en Specification?

La regla obtiene un nombre, se prueba por separado y se reutiliza. En lugar de repetir la misma condición en la API, en un reporte y en un proceso por lotes, se escribe una vez (VIP_DISCOUNT_SPEC) y se combina con &, | y ~. Si el umbral cambia, se modifica un solo lugar. Además, la Specification solo responde verdadero o falso, sin modificar el pedido, así que evaluarla no tiene efectos secundarios.

Pregunta 4. ¿Por qué los eventos se registran solo después de una transición válida?

Un Domain Event afirma que algo ya ocurrió. Si OrderShipped se registrara antes de validar la guía y la validación fallara, el sistema anunciaría un envío que no existe, y un suscriptor podría avisar al cliente de un paquete que nunca salió. Por eso el orden es: primero se validan las precondiciones, después se cambia el estado y al final se registra el evento; si una operación falla, no se registra el evento de una transición que no ocurrió. En E12, la segunda confirmación de O013 falla antes de llegar al registro y el pedido conserva un solo OrderConfirmed.

Pregunta 5. ¿Qué parte del código se puede reutilizar si se reemplaza FastAPI por otra interfaz?

Todo excepto app/api.py: app/domain.py (entidades, reglas, máquina de estados y eventos), app/specifications.py (descuentos), app/service.py (casos de uso), cargar_csv.py y las pruebas del dominio. Una aplicación de consola o de escritorio solo tendría que llamar a OrderService.confirm("O001") y mostrar el resultado o el mensaje de la excepción. ejecutar_escenarios.py ya es una segunda interfaz que usa el dominio sin FastAPI.

8. Conclusión general

En esta práctica implementamos el núcleo de negocio de TechStore en Python. Modelamos los pedidos con entidades y objetos de valor (Order, Customer, Money y OrderItem), expresamos las reglas de descuento como Specifications combinadas con AND, OR y NOT, controlamos el ciclo de vida del pedido con una máquina de estados que registra Domain Events y expusimos las operaciones mediante una API REST con FastAPI. Con los datos originales del paquete, los 15 escenarios E01–E15 cumplieron su objetivo y la suite de pytest terminó con 232 pruebas aprobadas.

El trabajo nos dejó cuatro aprendizajes principales. Validar los datos en el momento en que se crean los objetos evita que un importe negativo o un cliente mal definido avance por el sistema. Separar responsabilidades entre dominio, reglas, servicio y API permitió reutilizar la misma lógica desde el cargador, los escenarios, las pruebas y Swagger. Probar

las reglas en sus valores frontera, como 500.00 frente a 500.01, demostró lo que un solo escenario no alcanzaba a mostrar. Por último, ante una prueba fallida aprendimos a buscar primero la causa real antes de cambiar el código o el resultado esperado.

9. Conclusiones individuales

Lesly Adamaris Cotlame Nieto – 2226958

Al realizar esta actividad aprendí cómo se puede organizar y desarrollar la lógica de un sistema de manera más estructurada. Pude comprender mejor el uso de Specifications, las reglas de negocio, los cambios de estado y los Domain Events, además de aplicar una API con FastAPI y realizar pruebas con pytest. También aprendí que probar varias veces el proyecto y corregir errores es importante para asegurar que todo funcione correctamente. En general, esta actividad me ayudó a entender mejor cómo se construye y valida un sistema desde su lógica interna hasta su funcionamiento final.

Christopher Bethancourt Contreras – 2226948

En esta actividad aprendí a implementar la lógica de negocio de TechStore utilizando reglas de descuento, Specifications, una máquina de estados, Domain Events y una API REST con FastAPI. También comprendí la importancia de usar Decimal, separar las reglas y registrar los eventos solo cuando una transición es válida. Las pruebas y los errores que tuvimos me ayudaron a entender que validar y corregir el proyecto es fundamental para lograr un sistema confiable. En general, esta práctica me permitió comprender mejor cómo organizar y desarrollar un sistema de manera estructurada.

Francisco Mora Ruiz - 2226912

Con esta actividad aprendí un poco más sobre cómo se desarrolla y organiza un sistema de negocio. Pude conocer la importancia de tener reglas claras y de realizar pruebas para comprobar que el programa funcione correctamente. También comprendí que un proyecto necesita organización, revisión y corrección de errores para obtener buenos resultados. En general, esta actividad me permitió conocer mejor el proceso de desarrollo de un sistema y entender la importancia del trabajo en equipo para completar un proyecto.

10. Link del repositorio en github

https://github.com/leslynieto64-del/TechStore-U4-CLNEG-Equipo16